

REACT.JS

React Introduction:

- React is a JavaScript library used to build user interfaces, especially single-page applications.
- React is component-based, which means you build small, reusable pieces (called components) and combine them to create a full app.
- It helps in building reusable UI components and efficiently updates the UI using Virtual DOM
- It was developed by Facebook in 2013.

Why we use ReactJS?

- Easy to learn for anyone who knows JavaScript.
- React improves performance and code maintainability in large applications.
- Faster performance using Virtual DOM.
- Used in many popular apps like Facebook, WhatsApp Web, and Instagram

SPA:

- A Single Page Application (SPA) is a web application that:
- Loads one HTML page only once, and then dynamically updates content without reloading the entire page.

JSX:

- JSX stands for JavaScript XML.
- JSX allows writing HTML-like syntax inside JavaScript.
- It makes UI code more readable and expressive.
- JSX must return a single parent element and uses className instead of class.

Dom and Virtual DOM:

- DOM (Document Object Model) is the structure of the web page shown in the browser.
- In regular JavaScript, updating the DOM can be slow if done too many times.
- Virtual DOM is a copy of the real DOM used by React.
- React updates the Virtual DOM first, finds the changes, and updates only what's needed in the real DOM.

Differences:

Real DOM updates the entire structure, while Virtual DOM updates only the changed elements. Virtual DOM improves performance and efficiency.

Reconciliation:

Reconciliation is the process where React compares the new Virtual DOM with the previous one and updates only the necessary parts in the Real DOM

Components:

- ReactJS is all about components.
- ReactJS application is made up of multiple components, and each component has its own logic and controls.
- A component is like a small building block of a web page. Each component does one thing – like showing a button, a header, a card, or a form.
- In React, we write components using JavaScript + HTML-like code (called JSX).
- Components make your code cleaner, reusable, and easier to manage.
- Components allow dividing the UI into independent pieces.
- There are two types: Class Components and Functional Components.

Vanilla JS vs React JS:

Feature	Vanilla JavaScript	React JS
Approach	Manual DOM manipulation	Declarative UI
Code Style	Step-by-step Instructions	Describe UI state
Updates	You manually update UI	React updates automatically
Reusability	Hard (repeat code)	Easy (components)
Performance	Slower in large apps	Faster (Virtual DOM)
Maintainability	Becomes messy	Organized & scalable
Data Handling	You track everything	React manages UI state

CODE:

Vanilla JS

```
<!DOCTYPE html>
<html>
<body>
<h2 id="count">0</h2>
<button onclick="increase()">Increase</button>
<script>
let value = 0;
function increase() {
  value = value + 1;
  document.getElementById("count").innerText =
    value;
}
</script>
</body>
</html>
```

React JS

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => setCount(count +
        1)}>Increase</button>
    </div>
  );
}

export default Counter;
```

Rules of JSX:

1. Must return single parent element

```
return (  
  <div>  
    <h1>Hello</h1>  
    <p>Welcome</p>  
  </div>  
);
```

2. Use className instead of class

```
<h1 className="title">Hello</h1>
```

3. JavaScript inside { }

```
const name = "Nagamani";  
<h1>Hello {name}</h1>
```

4. Self-closing tags required

```
<hr />
```

Creating a React Application (Using Vite):

React applications need a development environment to run.
We use **Node.js** + **Vite** to quickly create and run React projects.

Vite is a fast build tool that helps in creating modern React applications with better performance and very quick startup time.

Step 1: Install Node.js

React requires Node.js to run JavaScript outside the browser.

1. Go to: <https://nodejs.org>
2. Download **LTS version**
3. Install normally (Next → Next → Finish)

Verify Installation

Open terminal / command prompt and type:

```
node -v  
npm -v
```

Step 2: Create React App using Vite

Open terminal inside the folder where you want the project and run:

```
npm create vite@latest my-app
```

my-app = your project name (you can change it)

Step 3: Select Project Options

You will be asked some questions:

1. Framework → **React**
 2. Variant → **JavaScript**
-

Step 4: Go Inside Project Folder

```
cd my-app
```

Step 5: Install Dependencies

```
npm install
```

Step 6: Run the React Application

```
npm run dev
```

You will see a local server link like:

<http://localhost:5173/>

Open it in browser

Your React app is now running.

Default Project Structure

my-app/

├─ node_modules/

├─ public/

├─ src/

| └─ App.jsx

| └─ main.jsx

| └─ assets/

├─ index.html
├─ package.json
└─ vite.config.js

Important Files

index.html → Root HTML file

main.jsx → Entry point of React app

App.jsx → Main component (UI starts here)

How React Renders the Page

1. index.html contains `<div id="root"></div>`
2. main.jsx connects React to root
3. App.jsx displays the UI

Class Component:

A class component is created using ES6 class syntax and extends Component.

```
import React, { Component } from 'react'

class App extends Component {
  render() {
    return (
      <div>
        <h1>Hello World</h1>
      </div>
    )
  }
}

export default App
```

Explanation

- class App extends Component → creating a class component
- render() → mandatory method that returns JSX
- Used before React Hooks were introduced

Functional Component:

A functional component is a normal JavaScript function that returns JSX.

```
import React from 'react'

function App() {
  return (
    <div>
      <h1>Hello World</h1>
    </div>
  )
}

export default App
```

- Just a JavaScript function
- No render() method required
- Simpler and easier to understand
- Most commonly used in modern React

Two Functional Components in one React File

Example:

```
function Header() {
  return <h1>This is Header</h1>;
}
```

```
function Footer() {
  return <h3>This is Footer</h3>;
}
```

```
function App() {
  return (
    <div>
      <Header />
      <Footer />
    </div>
  );
}

export default App;
```

Event handler:

```
function ButtonExample() {  
  function handleClick() {  
    alert("Button Clicked!");  
  }  
  
  return (  
    <div>  
      <h1>Hello React</h1>  
      <button onClick={handleClick}>Click Me</button>  
    </div>  
  );  
}  
  
export default ButtonExample;
```

Props in React

Props stand for **Properties**.

Props are used to pass data from one component to another component (usually from parent to child).

Think of props like arguments passed to a function.

Why We Use Props?

- To make components reusable
- To send dynamic data
- To customize UI components

Example: Same component → different output

Basic Example

Step 1: Create Child Component

Student.jsx

```
function Student(props) {  
  return (  
    <div>  
      <h2>Name: {props.name}</h2>  
      <p>Course: {props.course}</p>  
    </div>  
  )  
}  
export default Student
```

Step 2: Send Data from Parent Component

App.jsx

```
import Student from "./Student"  
function App() {  
  return (  
    <div>  
      <Student name="Ravi" course="CSE" />  
      <Student name="Anu" course="ECE" />  
      <Student name="Kiran" course="IT" />  
    </div>  
  )  
}  
export default App
```

Output

Name: Ravi Course: CSE

Name: Anu Course: ECE

Name: Kiran Course: IT

Same component → different data → reusable component

Destructuring Props (Recommended Method)

Instead of writing `props.name`, we can directly extract values.

```
function Student({ name, course }) {  
  return (  
    <div>  
      <h2>Name: {name}</h2>  
      <p>Course: {course}</p>  
    </div>  
  )  
}
```

This makes code cleaner and easier to read.

Important Points of Props

1. Props are read-only (immutable)
2. Props flow from Parent → Child only (one-way data flow)
3. Props help create reusable components

State in React

State is a built-in object used to store data that can change over time inside a component.

When state changes → React automatically updates the UI.

In simple words:

State is a component's **own memory**.

Why We Use State?

- To store dynamic values
- To handle user interaction
- To update UI automatically

Examples:

- Counter
- Form input
- Show/Hide password

- Like button

Functional Component State (useState Hook)

In modern React, we use **useState Hook** to create state.

```
import { useState } from "react"

function Counter() {
  const [count, setCount] = useState(0)

  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => setCount(count + 1)}>Increase</button>
    </div>
  )
}

export default Counter
```

Explanation

useState(0) → initial value is 0

count → current state value

setCount → function used to update state

When button is clicked:

1. count value changes
2. React re-renders component
3. UI updates automatically

Example

```
import { useState } from "react";

function App() {
  const [name, setName] = useState("Nagamani");
  const [count, setCount] = useState(0);
```

```

return (
  <div>
    <h1>Hello {name}</h1>
    <h2>Count: {count}</h2>

    <button onClick={() => setCount(count + 1)}>
      Increase
    </button>

    <button onClick={() => setName("FSD Trainer")}>
      Change Name
    </button>
  </div>
);
}

export default App;

```

Class Component State (Old Method)

```

import React, { Component } from "react"
class Counter extends Component {
  constructor() {
    super()
    this.state = { count: 0 }
  }
  increase = () => {
    this.setState({ count: this.state.count + 1 })
  }
  render() {

```

```

return (
  <div>
    <h2>{this.state.count}</h2>
    <button onClick={this.increase}>Increase</button>
  </div>
)
}
}

```

export default Counter

Props vs State

Props	State
Passed from parent	Managed inside component
Read-only	Can be changed
Used for communication	Used for dynamic data
External data	Internal data

Expressions in React

```

function App() {
  const name = "Full Stack Development";
  return (
    <div>
      <h1>Hello, {name}</h1>
      <p>Sum of 2 + 3 = {2 + 3}</p>
    </div>
  );
}

```

export default App;

Conditional Rendering in React

Conditional Rendering means **showing different UI based on a condition**.

React allows us to display elements depending on state, props, or any logical condition.

Example:

- Login / Logout button
 - Show profile only after login
 - Show "Loading..." while fetching data
 - Show "No Data Found"
-

Why We Use Conditional Rendering?

- To show/hide elements
 - To display messages dynamically
 - To control UI based on user action
 - To improve user experience
-

1. Using if-else Statement (with Props)

App.js (Parent)

```
import Login from "./Login";
```

```
function App() {  
  return <Login isLoggedIn={true} />;  
}
```

```
export default App;
```

Login.js (Child)

```
function Login(props) {  
  if (props.isLoggedIn) {  
    return <h2>Welcome User</h2>;  
  } else {  
    return <h2>Please Login</h2>;  
  }  
}
```

```
export default Login;
```

Explanation

If isLoggedIn = true → Welcome User
If false → Please Login

2. Using Ternary Operator (with Props)

App.js

```
import Login from "./Login";

function App() {
  return <Login isLoggedIn={false} />;
}

export default App;
```

Login.js

```
function Login(props) {
  return (
    <div>
      <h2>
        {props.isLoggedIn ? "Welcome User" : "Please Login"}
      </h2>
    </div>
  );
}

export default Login;
```

Explanation

condition ? trueValue : falseValue

3. Using Logical AND (&&) Operator (with Props)

App.js

```
import Notification from "./Notification";

function App() {
  return <Notification hasMessage={true} />;
}

export default App;
```

Notification.js

```
function Notification(props) {
  return (
    <div>
```

```

    {props.hasMessage && <h3>You have a new message</h3>}
  </div>
);
}

```

export default Notification;

Explanation

If hasMessage = true → message appears

If false → nothing appears

4. Using Variable for Complex Logic (if–else if) with Props

When we have multiple conditions.

App.js

```
import UserStatus from "./UserStatus";
```

```

function App() {
  return <UserStatus role="admin" />;
}

```

export default App;

UserStatus.js

```

function UserStatus(props) {
  let message;

  if (props.role === "admin") {
    message = "Welcome Admin";
  } else if (props.role === "user") {
    message = "Welcome User";
  } else {
    message = "Guest Access";
  }

  return <h2>{message}</h2>;
}

```

export default UserStatus;

Explanation

role = "admin" → Welcome Admin

role = "user" → Welcome User

else → Guest Access

Important Points:

1. if-else → Basic condition
2. Ternary → Most common inside JSX
3. && operator → Show only if TRUE
4. Variable + if-else if → Multiple conditions

Iterative Rendering in React (map())

Iterative Rendering means displaying multiple elements dynamically using data.

In React, we use the **map() function** to loop through an array and render UI elements.

Why We Use map()?

- To display lists of data
 - To show API data (users, products, posts)
 - To avoid writing repeated HTML
 - To create dynamic UI
-

Basic Example

```
function Students() {  
  const names = ["Ravi", "Anu", "Kiran", "Meena"]  
  return (  
    <div>  
      {names.map((name) => (  
        <h3>{name}</h3>  
      ))}  
    </div>  
  )  
}  
export default Students
```

Output

Ravi
Anu
Kiran
Meena

React creates multiple elements automatically.

Rendering Objects (Most Important Case)

```
function Students() {  
  const students = [  
    { id: 1, name: "Ravi", course: "CSE" },  
    { id: 2, name: "Anu", course: "ECE" },  
    { id: 3, name: "Kiran", course: "IT" }  
  ]  
  
  return (  
    <div>  
      {students.map((stu) => (  
        <div key={stu.id}>  
          <h3>{stu.name}</h3>  
          <p>{stu.course}</p>  
        </div>  
      ))}  
    </div>  
  )  
}
```

export default Students

What is key in React?

key is a special attribute used to uniquely identify elements in a list.

React uses key to:

- Track changes

- Update only changed items
 - Improve performance
-

Example

```
const fruits = ["Apple", "Mango", "Banana"];
```

```
{fruits.map((fruit, index) => (  
<h3 key={index}>{fruit}</h3>  
))}
```

Here:

- fruit → value
- index → position
- key → uniquely identifies element

Best Practice Example (Using ID)

```
const users = [  
  { id: 1, name: "Reshma" },  
  { id: 2, name: "Anu" }  
];
```

```
{users.map(user => (  
<h3 key={user.id}>{user.name}</h3>  
))}
```

React Hooks

Hooks are special functions in React that allow functional components to use React features like state and lifecycle methods.

Before Hooks → only Class Components could use state

After Hooks → Functional Components can also use state

Hooks were introduced in React 16.8

Why Hooks?

- Make code simpler
 - Avoid class components
 - Reuse logic easily
 - Improve readability
 - Used in all modern React apps
-

Rules of Hooks

1. Hooks must be called at top level
 2. Hooks cannot be used inside loops or conditions
 3. Hooks can only be used inside functional components
-

useState Hook

useState is the most important Hook in React.

It is used to create and manage state in functional components.

It allows components to remember values and update UI automatically.

Syntax

```
const [state, setState] = useState(initialValue)
```

Meaning

state → current value

setState → function to update value

initialValue → starting value

Example 1: Counter

```
import { useState } from "react"

function Counter() {
  const [count, setCount] = useState(0)

  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>

      <button onClick={() => setCount(count - 1)}>
        Decrease
      </button>
    </div>
  )
}
```

```
    </div>
  )
}
export default Counter
```

What Happens Here?

1. Initial value = 0
 2. Button clicked
 3. State updates
 4. React re-renders UI automatically
-

Example 2: Input Box (Most Important Example)

```
import { useState } from "react"

function Form() {
  const [name, setName] = useState("")
  return (
    <div>
      <input
        type="text"
        placeholder="Enter name"
        onChange={(e) => setName(e.target.value)}
      />
      <h3>Hello {name}</h3>
    </div>
  )
}
export default Form
```

Multiple States

```
const [name, setName] = useState("")
const [age, setAge] = useState(0)
```

```
const [city, setCity] = useState("")
```

You can create multiple state variables in one component.

useEffect Hook in React

useEffect is a Hook used to perform **side effects** in a functional component.

Side effects are operations that happen outside the UI rendering.

Examples:

- Fetching API data
 - Timer / setInterval
 - Updating document title
 - Event listeners
-

Why We Need useEffect?

React components re-render many times.

We cannot run API calls or heavy operations directly inside JSX.

So, React provides useEffect to handle those operations safely.

Syntax

```
useEffect(() => {  
  // code to run  
}, [dependency])
```

Parts

Function → code to execute

Dependency Array → controls when effect runs

Types of useEffect

1. Run on Every Render

```
import { useState, useEffect } from "react"
```

```
function Example() {  
  const [count, setCount] = useState(0)
```

```
useEffect(() => {  
  console.log("Component Rendered")  
})  
return (  
  <button onClick={() => setCount(count + 1)}>  
    {count}  
  </button>  
)  
}
```

Runs after every render

2. Run Only Once (Most Common)

Empty dependency array

```
import { useEffect } from "react"
```

```
function Example() {  
  useEffect(() => {  
    console.log("Runs only once after first render")  
  }, [])  
  return <h2>Hello</h2>  
}
```

Used for:

- API calls
 - Initial loading
-

3. Run When Value Changes

```
import { useState, useEffect } from "react"
```

```
function Example() {  
  const [count, setCount] = useState(0)
```

```

useEffect(() => {
  console.log("Count changed:", count)
}, [count])
return (
  <button onClick={() => setCount(count + 1)}>
    {count}
  </button>
)
}

```

Runs only when count updates

API Fetch Example (Important)

```

import { useState, useEffect } from "react"

function Users() {
  const [users, setUsers] = useState([])
  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(res => res.json())
      .then(data => setUsers(data))
  }, [])
  return (
    <div>
      {users.map(user => (
        <h3 key={user.id}>{user.name}</h3>
      ))}
    </div>
  )
}

export default Users

```

Important Points

- Runs after rendering
- Used for API calls
- Can control execution using dependency array
- Can return cleanup function

React API Handling

API Handling means connecting React application with backend server to **send or receive data**.

Most modern applications do not store data inside the UI.
They fetch data from a server (API).

Examples:

- Login system
 - Products list
 - Users list
 - Weather app
-

What is an API?

API (Application Programming Interface) is a bridge between frontend and backend.

React (Frontend) ↔ API ↔ Database (Backend)

React sends request → Server returns data → React displays data

Fetch API

React uses JavaScript **fetch()** to request data from server.

Basic Syntax

```
fetch("API_URL")  
  .then(response => response.json())  
  .then(data => console.log(data))
```

API Integration in React

We must call API **after component loads**
So we use `useEffect`

Fetching Data from API


```
import { useState, useEffect } from "react"

function Users() {
  const [users, setUsers] = useState([])

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(res => res.json())
      .then(data => setUsers(data))
  }, [])

  return (
    <div>
      <h2>User List</h2>
      {users.map(user => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  )
}

export default Users
```

Flow Explanation

1. Component loads
2. useEffect runs
3. API request sent
4. Data received
5. State updated
6. UI re-renders

Loading State (Very Important)

Real apps take time to fetch data.

```
import { useState, useEffect } from "react"
```

```
function Users() {  
  const [users, setUsers] = useState([])  
  const [loading, setLoading] = useState(true)  
  
  useEffect(() => {  
    fetch("https://jsonplaceholder.typicode.com/users")  
      .then(res => res.json())  
      .then(data => {  
        setUsers(data)  
        setLoading(false)  
      })  
  }, [])  
  
  if (loading) return <h2>Loading...</h2>  
  
  return (  
    <div>  
      {users.map(user => (  
        <p key={user.id}>{user.name}</p>  
      ))}  
    </div>  
  )  
}
```

Error Handling

```
import { useState, useEffect } from "react"

function Users() {
  const [users, setUsers] = useState([])
  const [error, setError] = useState(null)

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(res => res.json())
      .then(data => setUsers(data))
      .catch(() => setError("Failed to load data"))
  }, [])

  if (error) return <h2>{error}</h2>

  return (
    <div>
      {users.map(user => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  )
}
```

Important Points

- API calls should be inside useEffect
- Store data using useState
- Always handle loading and error
- UI updates automatically after data comes

Forms in React – **Controlled Components**

In React, form elements like input, textarea, and select are handled using **state**.

A **Controlled Component** is a form element whose value is controlled by React state.

Why Controlled Components?

- React keeps form data
 - Easy validation
 - Dynamic updates
 - Single source of truth
-

Basic Input Example

```
import { useState } from "react"

function Form() {
  const [name, setName] = useState("")
  return (
    <div>
      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Enter name"
      />
      <h3>Your Name: {name}</h3>
    </div>
  )
}

export default Form
```

Explanation

value={name} → React controls input

onChange → updates state

State changes → UI updates

Input Handling

Capturing user input using onChange event and storing it in state.

Example:

```
import { useState } from "react"
```

```
function Form() {
```

```
  const [formData, setFormData] = useState({
```

```
    name: "",
```

```
    email: ""
```

```
  })
```

```
  const handleChange = (e) => {
```

```
    setFormData({
```

```
      ...formData,
```

```
      [e.target.name]: e.target.value
```

```
    })
```

```
  }
```

```
  return (
```

```
    <div>
```

```
      <input
```

```
        type="text"
```

```
        name="name"
```

```
        placeholder="Name"
```

```
        onChange={handleChange}
```

```
      />
```

```
      <input
```

```
        type="email"
```

```
        name="email"
```

```
        placeholder="Email"
```

```
        onChange={handleChange}
```

```

    />
    <h3>{formData.name}</h3>
    <h3>{formData.email}</h3>
  </div>
)
}
export default Form

```

Form Submission

Handling form submit using onSubmit and preventing page reload using preventDefault().

Example:

```

import { useState } from "react"
function Form() {
  const [name, setName] = useState("")

  const handleSubmit = (e) => {
    e.preventDefault()
    alert("Form Submitted: " + name)
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />
      <button type="submit">Submit</button>
    </form>
  )
}

```

export default Form
